

OPERATING SYSTEMS PROJECT REPORT

MiniOS: A Command Line Operating System Simulator

Process Management, Round Robin Scheduling, Memory Allocation, and Semaphore
Synchronization in Java

Quinn Smith
CS 3230, Section 01, Spring 2024

Executive Summary

An operating system hides a great deal of machinery behind a few simple commands, and building a small one is one of the best ways to understand it. MiniOS is a command line operating system simulator written in Java by a team of three for a senior operating systems course. It models four of the core services of a real operating system: process management, round robin scheduling, contiguous memory allocation, and synchronization with a semaphore.

The program is made up of five source files totaling 472 lines, including documentation comments. A user creates processes, lists them, assigns blocks of memory from a 100 unit memory array, and starts a scheduler that runs every ready process in turns of two seconds. A binary semaphore guards each turn, and memory is returned automatically when a process finishes. The processes do not execute real programs. Each one is represented by a process control block and a countdown timer, which keeps the focus on the operating system mechanisms themselves.

Key finding: In a recorded demonstration, five processes with run times of 14, 6, 9, 5, and 15 seconds finished in 26 time slices and 52 seconds. Every process received its first turn within 8 seconds, compared with an average wait of 19.4 seconds if the same processes had run first come first served. The test also revealed a real defect: memory given to PID 0 is stored as the value 0, which the memory manager treats as free.

This report describes the design and implementation of each component, presents the behavior observed in the recorded session, and documents the limitations of the current version. It is candid about those limitations because understanding why a simple design breaks is as instructive as understanding why it works. Concepts are tied to standard operating systems references (Silberschatz et al., 2018; Tanenbaum & Bos, 2015).

1. Introduction

An operating system has three jobs that every other program depends on. It decides which process uses the processor, it decides which process owns which part of memory, and it coordinates processes so that shared resources are not corrupted by simultaneous access. Textbooks describe these jobs in the abstract, but the ideas become much clearer when a student has to write the data structures and loops that carry them out (Silberschatz et al., 2018).

MiniOS was built to make those ideas concrete. It is deliberately small. It runs inside a single Java program, uses a text prompt as its only interface, and simulates processes with timers instead of running real code. Even so, it contains working versions of a process control block, a round robin scheduler, a first fit memory allocator, and a counting semaphore, and the interaction between them follows the same logic a full operating system would use.

The goals of the project were to implement each of these components cleanly, to connect them so that one action affects the others (for example, a finished process releases its memory), and to document the behavior honestly, including where the simulation falls short of a real system.

2. Background

2.1 Processes and the Process Control Block

A process is a program in execution. The operating system keeps one record for each process, called a process control block (PCB), that stores everything needed to manage it, including its identifier, its current state, and information about the resources it holds (Silberschatz et al., 2018). The state of a process moves through a small set of values. A process that is waiting for the processor is ready, a process that currently holds the processor is running, and a process that is waiting for an event such as input or output is blocked (Tanenbaum & Bos, 2015). MiniOS uses exactly these three states.

2.2 Round Robin Scheduling

Round robin is a preemptive scheduling algorithm designed for time sharing. Each ready process receives the processor for at most a fixed interval called the time quantum. If the process has not finished when the quantum expires, it returns to the end of the ready queue and the next process runs (Silberschatz et al., 2018). The algorithm is simple and fair, and it gives every process a prompt first response. Its cost is that average turnaround time can be worse than with other algorithms, and the choice of quantum matters: a very large quantum behaves like first come first served, and a very small one wastes time switching between processes.

2.3 Contiguous Memory Allocation

In contiguous allocation, each process receives one unbroken block of memory. When a request arrives, the system must pick a free block, often called a hole, large enough to hold it. The first fit strategy takes the first hole that is big enough, and it is popular because it is fast. Over time, repeated allocation and release leaves many small holes scattered through memory, a problem known as external fragmentation (Silberschatz et al., 2018).

2.4 Semaphores

A semaphore is an integer variable that can be changed only through two atomic operations. The wait operation decrements the value and blocks the caller if the value is not positive, and the signal operation increments the value and wakes a waiting process. Dijkstra introduced the idea in 1965 as a way to coordinate cooperating sequential processes (Dijkstra, 1965). A semaphore with an initial value of one is called a binary semaphore and works as a mutual exclusion lock. Java does not need a separate semaphore implementation for this purpose, but its built in monitor methods, wait and notify, provide the primitives from which one can be built (Oracle, n.d.a).

3. Objectives and Requirements

3.1 Project Objectives

- Provide a command line interface through which a user can create processes, inspect them, allocate memory, and start scheduling.
- Represent each process with a process control block that records its identifier, name, state, activity, and remaining run time.

- Schedule ready processes with round robin and a fixed time quantum until all of them finish.
- Simulate a fixed size memory, allocate contiguous blocks to processes, and release them automatically when a process completes.
- Protect each scheduled time slice with a semaphore so that only one process runs at a time.

3.2 Command Interface

Command	Behavior	Handled by
create [name]	Creates a process with the given name, a new PID, and a random run time of 5 to 15 seconds	ProcessManager
ps	Lists every process with its PID, name, state, and active flag	ProcessManager
schedule	Runs all ready processes with round robin until every process has finished	ProcessManager
alloc [pid] [size]	Allocates a contiguous block of the requested size to an existing process	MemoryManager
mem	Prints the memory layout, ten units per row	MemoryManager
help	Prints the list of commands	Main
exit	Ends the program	Main

Table 3.1. MiniOS commands.

4. System Design

4.1 Architecture

MiniOS is organized into five classes with clearly separated responsibilities. The Main class owns the user interface and delegates every command. ProcessManager owns the list of processes and the scheduler, and it is the only class that talks to both the PCB and Semaphore classes. MemoryManager is independent of the process model and knows processes only by their numeric PID. Figure 4.1 shows how the pieces connect, and Table 4.1 summarizes each file.

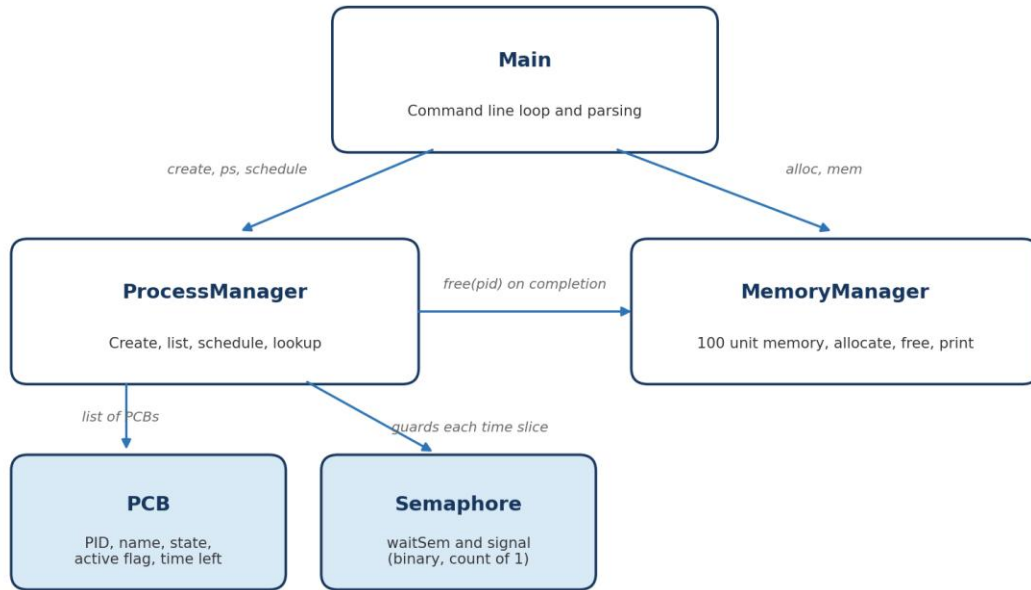


Figure 4.1. MiniOS architecture and the main calls between classes.

Class	Lines	Responsibility
Main	102	Prompt loop, command parsing, usage messages, and input validation for alloc
ProcessManager	113	Process list, PID assignment, process creation and listing, the round robin scheduler, and PID lookup
PCB	117	Process control block with the State enumeration (READY, RUNNING, BLOCKED) and accessor methods
MemoryManager	90	A 100 unit memory array, first fit allocation, release by PID, and printing
Semaphore	50	A counting semaphore built on Java monitor methods

Table 4.1. Source files. Line counts include documentation comments.

4.2 Process Model

Each process is a PCB object with five fields, listed in Table 4.2. Because MiniOS simulates processes instead of running them, the only work a process performs is counting down its remaining time.

Field	Type	Meaning
pid	int	Unique identifier, assigned in order starting at 0
name	String	Name supplied by the user at creation
state	State	READY, RUNNING, or BLOCKED
active	boolean	True until the process has finished its work
remainingTime	int	Seconds of work left, set to a random value from 5 to 15 at creation

Table 4.2. Fields of the process control block.

A new process starts in the READY state and is active. During scheduling it moves to RUNNING for one time slice. If work remains it returns to READY, and if the work is done its active flag is cleared and its memory is released.

4.3 Memory Model

Memory is a Java integer array with 100 units. A value of 0 marks a free unit, and any other value is the PID of the process that owns it. This makes the layout easy to print and easy to search, but it creates a conflict when a process has PID 0, as Section 6 shows.

4.4 Design Decisions

- **Real time simulation.** The scheduler uses `Thread.sleep` to wait out each time slice (Oracle, n.d.b). A two second quantum therefore takes two real seconds, which lets the user watch the algorithm work.
- **Central ownership of processes.** Only `ProcessManager` changes process state, so scheduling logic is in one place.
- **Loose coupling to memory.** `ProcessManager` receives the `MemoryManager` as a parameter to `schedule`, so the two classes share no permanent reference.

5. Implementation

5.1 Command Loop

Main reads a line with a `Scanner`, trims it, splits it on whitespace, and uses a switch statement on the first token. Each command checks its argument count and prints a usage message when it is wrong. The `alloc` command parses its two arguments as integers inside a try block that catches `NumberFormatException`, and it confirms that the PID belongs to an existing process before calling the memory manager. Unknown commands produce a short hint to type `help`.

5.2 Process Creation

The `create` command calls `ProcessManager`, which assigns the next PID from a counter that starts at zero, draws a random run time from 5 to 15 seconds, builds a PCB in the `READY` state, and adds it to a list. It then prints the PID, name, and run time.

5.3 The Round Robin Scheduler

The scheduler uses a quantum of two seconds and repeatedly sweeps through the process list in PID order. Because each sweep visits processes in the same order, the list itself acts as the ready queue. For each sweep it performs the following steps:

1. Skip any process that is not both `READY` and active. If none qualifies in a full sweep, print that all processes are complete and stop.
2. Call `waitSem` on the semaphore to enter the critical section.
3. Set the process to `RUNNING` and print its name, PID, and remaining time.
4. Sleep for the quantum, then subtract the quantum from the remaining time.
5. If no time remains, clear the active flag, call `free` on the memory manager for that PID, and print a completion message. Otherwise return the process to `READY`.
6. Call `signal` to leave the critical section.

5.4 The Semaphore

The Semaphore class holds an integer count and declares both operations as synchronized, so only one thread can change the count at a time. The wait operation loops while the count is zero or less, calling Java's wait method each time, and then decrements the count. The signal operation increments the count and calls notify to wake one waiting thread (Oracle, n.d.a). ProcessManager creates its semaphore with an initial count of one, which makes it a binary semaphore that enforces mutual exclusion around each time slice.

5.5 The Memory Manager

Allocation is a first fit search. The method scans the array from the start while tracking the beginning and length of the current run of free units. A used unit resets the run. When the run reaches the requested size the scan stops, the PID is written into every unit of the run, and the method reports success. If the scan reaches the end without finding a long enough run, it reports that there is not enough memory. Releasing memory scans the entire array and sets every unit owned by the PID back to zero. Printing writes the array as comma separated values with ten units per line.

Table 5.1 shows the five allocation requests from the recorded session and where first fit placed each block.

Request	Size	Result	Placement
alloc 0 10	10	Allocated	Units 0 to 9, but stored as 0 and so not visible
alloc 1 20	20	Allocated	Units 0 to 19, first free run found (overwrote PID 0's block)
alloc 2 25	25	Allocated	Units 20 to 44
alloc 3 30	30	Allocated	Units 45 to 74
alloc 4 50	50	Rejected	Only 25 free units remained
alloc 4 20	20	Allocated	Units 75 to 94

Table 5.1. Allocation requests and first fit placement in the recorded session.

6. Testing and Observations

6.1 Test Environment and Method

The system was tested by recording one continuous session of about two minutes and sixteen seconds in Windows PowerShell. The source was compiled with the javac compiler, and the compiled classes use class file version 61, which corresponds to Java 17. The session followed the full workflow a user would follow: view help, inspect empty memory and an empty process list, create five processes, allocate memory, inspect memory, run the scheduler to completion, and inspect memory again. This was a functional demonstration of a single scenario and not an automated test suite.

6.2 Process Creation and Memory

Five processes named 1 through 5 were created with PIDs 0 through 4 and run times of 14, 6, 9, 5, and 15 seconds. The ps command listed all five as READY and active. After the allocations in Table 5.1, the memory display matched the layout in Figure 6.1. Before any allocation, and again after scheduling finished, memory showed all zeros.

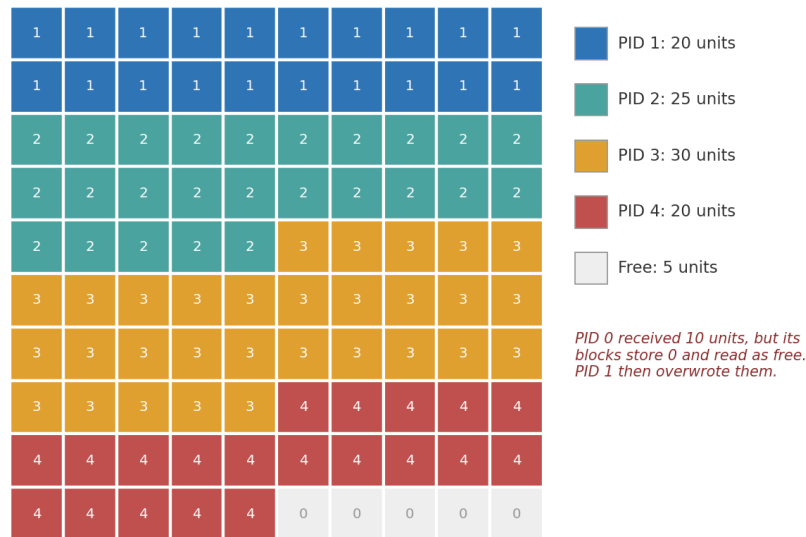


Figure 6.1. Memory layout before scheduling. Each square is one memory unit labeled with the owning PID.

6.3 Scheduling Results

The scheduler printed 26 running lines, so the five processes needed 26 time slices and 52 seconds of simulated time. The order of turns matched round robin exactly: five turns in PID order, then another sweep, and so on, with finished processes dropping out of later sweeps. Figure 6.2 reconstructs the timeline from the printed output.

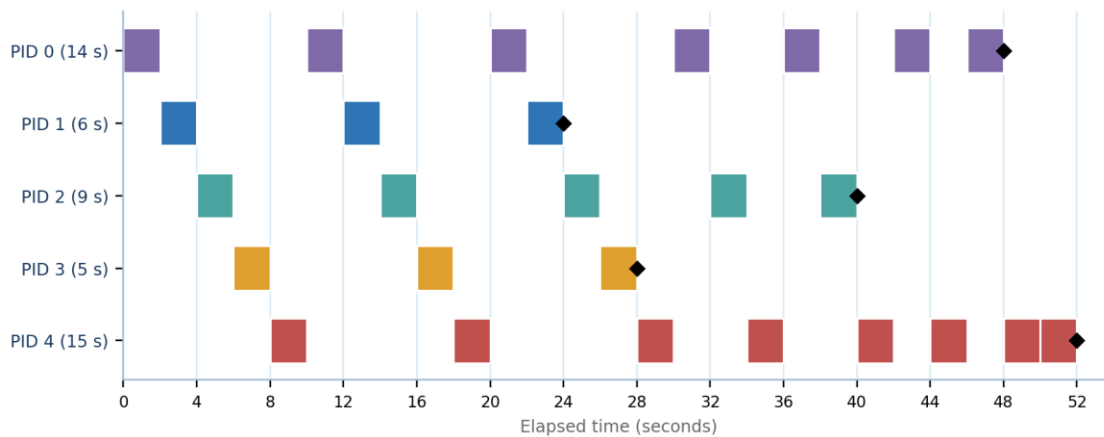


Figure 6.2. Round robin timeline. Each block is one two second slice, and a diamond marks completion.

PID	Run time	Time slices	First turn at	Completed at	Completion order
0	14 s	7	0 s	48 s	4th
1	6 s	3	2 s	24 s	1st
2	9 s	5	4 s	40 s	3rd
3	5 s	3	6 s	28 s	2nd
4	15 s	8	8 s	52 s	5th

Table 6.1. Per process results, derived from the printed trace.

6.4 Comparison with First Come First Served

Because all five processes were ready when scheduling began, the trace allows a direct comparison with a first come first served order using the same run times. Table 6.2 shows the result.

Measure	Round robin (observed)	First come first served
Average time to first turn	4.0 s	19.4 s
Average completion time	38.4 s	29.2 s
Total time to finish all processes	52 s	49 s

Table 6.2. Round robin compared with first come first served. First come first served values are calculated from the same run times and are not from the session.

The results show the standard trade off described in the literature (Silberschatz et al., 2018). Round robin gave every process a prompt first turn, which is what makes an interactive system feel responsive, but it raised the average completion time because long processes were interrupted repeatedly and short ones were stretched across sweeps. The three extra seconds in the total are explained in Section 7.

6.5 Memory Release

After the scheduler reported that all processes were complete, the mem command showed memory entirely zero again. This confirms that the completion path in the scheduler correctly calls the memory manager for each finished process.

7. Limitations and Known Issues

Reviewing the source and the recorded session identified several issues. The first is the most serious because it changed the outcome of the demonstration.

Impact of the PID 0 defect: Had PID 0's 10 units been tracked correctly, the four successful allocations would have consumed 85 of the 100 units, leaving 15. The final request for 20 units by PID 4 would then have been rejected, but in the session it succeeded.

#	Issue	Description and effect	Severity	Source
1	PID 0 collides with free memory	The first process receives PID 0, and 0 also marks a free unit. Blocks allocated to PID 0 cannot be told apart from free memory and are overwritten by the next allocation.	High	Session
2	Finished processes show RUNNING	A completed process has its active flag cleared but its state is never reset, so ps would show State RUNNING with Active false.	Medium	Code
3	Memory can be allocated to finished processes	The alloc command only checks that the PID exists. Memory given to a process that has already completed is never released, because release happens only at completion.	Medium	Code
4	Full quantum is always charged	The scheduler subtracts the whole quantum even when less time remains. Processes with odd run times (9, 5, and 15) each lose one second, so 49 seconds of work took 52.	Low	Session
5	Semaphore never contends	The scheduler runs on one thread, so the semaphore never blocks. Its header comment also says two processes may run at once, while it is created with a count of one.	Low	Code

#	Issue	Description and effect	Severity	Source
6	BLOCKED state is unused	The state is defined but no code sets it, so there is no simulation of waiting for input or output.	Low	Code
7	Scheduler blocks the prompt	The schedule command does not return until all processes finish, so nothing can be created or allocated during a run that may last a minute or more.	Low	Session
8	Limited input and memory control	Zero or negative allocation sizes are not rejected, there is no command to free memory or remove a process, memory is never compacted, and the free method contains an unused variable.	Low	Code

Table 7.1. Known issues. The Source column shows whether an issue appeared in the recorded session or was found by reading the code.

8. Future Work

- Use a separate value, such as negative one, to mark free memory so that PID 0 is handled correctly, or start PIDs at one.
- Add a terminated state and reset a process's state on completion, and reject allocation requests for processes that are not active.
- Simulate blocking by giving processes occasional input or output waits that move them to the BLOCKED state and back.
- Run each process on its own Java thread so the semaphore actually controls contention, then add a demonstration such as the producer consumer problem.
- Charge only the time actually used in the last slice, and allow the quantum to be configured from the prompt.
- Add best fit and worst fit strategies, a memory compaction command, and a command to free memory or kill a process, then compare fragmentation across strategies.
- Report scheduling statistics automatically after each run, and add priority scheduling for comparison with round robin.
- Write unit tests for the memory manager and scheduler so that regressions such as the PID 0 defect are caught automatically.

9. Conclusion

MiniOS demonstrates the central ideas of an operating system in under five hundred lines of Java. Its process control block, round robin scheduler, first fit memory allocator, and semaphore work together, and the recorded session showed them behaving as the theory predicts: every process received a prompt first turn, finished processes released their memory, and the trade off between responsiveness and average completion time appeared in the numbers.

The project also showed how small design choices have large consequences. Using 0 as both a valid process identifier and the marker for free memory silently changed the outcome of an allocation test. Finding and explaining that kind of defect, instead of only building the happy path, is the main lesson of the project. The planned improvements in Section 8 would remove the known defects and extend the simulator toward a more realistic model.

References

- Dijkstra, E. W. (1965). *Cooperating sequential processes* (EWD123). Technological University, Eindhoven.
<https://www.cs.utexas.edu/~EWD/transcriptions/EWD01xx/EWD123.html>
- Oracle. (n.d.a). *Object (Java SE 17 API specification): wait and notify methods*.
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html>
- Oracle. (n.d.b). *Thread (Java SE 17 API specification): sleep method*.
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.